

Java - 给 Spring Boot 服务配置通用脚本

Original YZhou 阿洋聊大数据 2025年3月20日 13:16 北京

Java - 给 Spring Boot 服务配置通用脚本

背景

在学习和工作中接触了一些开源的 Java 项目，它们绝大多数在部署服务的根目录下存在 `bin` 子目录用来存放一些 `sh` 脚本对服务进行 `启停操作`，存在 `conf` 子目录用来存放一些配置文件对服务进行配置参数管理，例如 `kafka`, `flink`, `rocketmq`, `streampark` 等项目，用起来确实很方便，下面列举一些示例：

下面是 `RocketMQ` 的部署目录截图：

下面是 `Kafka` 的部署目录截图

该篇 Blog 主要介绍的内容是基于 `Spring Boot` 框架开发的服务如何也像上面优秀的开源项目一样，有 `bin` 目录提供脚本进行服务启停，有 `conf` 目录提供配置参数管理。

有与没有之间

从 <https://start.spring.io/> 网站构建 Spring Boot `jar` 项目时，并不存在 `bin`，`conf` 目录，下面是初始化项目的目录结构：

这部分与 `Playframework for Scala` 框架非常不同，博主的 Blog (<http://xinzhuixiansheng.com/>) 是使用 Play for Scala 开发的，下面是它的目录结构：

```
[root@blogvm blog-play-1.0-SNAPSHOT]# ls
bin  conf  lib  logs  README.md  RUNNING_PID  share
```

这部分 **Play for Scala** 做的比 Spring Boot 好太多，**好的习惯是成功的一部分**。

在 Spring Boot **jar** 项目中，若不做特殊处理，配置文件会在打包项目时集成到 **jar** 文件内部，这不利于我们去手动调整配置参数，可能我们会重新打包服务再部署，或者你将服务的配置参数集成到 **配置中心** 这样的服务中，或者集成到 MySQL，这些也能简化我们调整参数的成本，但对于一些业务量，暂不需要双活等情况下，这会增加一些开发成本。但我们还是可以通过一些其他手段来解决它的，下面会介绍。

当你通过某些手段提高一些事情的效率时，你可以比别人有更多的时间做其他事。

Spring Boot 集成 bin, conf

我们先看下完整示例模板的结构，（这个项目模板是我写微信小程序的后台服务。）

Project ▾



- ▾ **turntable-springboot** E:\Code\Yzhou\turntable-springboot
 - > .idea
 - ▾ **distribution**
 - ▾ bin
 - shutdown.cmd
 - shutdown.sh
 - startup.cmd
 - startup.sh
 - ▾ conf
 - application.yaml
 - turntable-server-logback.xml
 - ▾ target
 - archive-tmp
 - > maven-archiver
 - > turntable-server-1.0-SNAPSHOT
 - turntable-server.jar
 - turntable-server-1.0-SNAPSHOT.tar.gz
 - turntable-server-1.0-SNAPSHOT.zip
 - assembly.xml
 - pom.xml
 - ▾ framework [**turntable-framework**]
 - > cache [**turntable-cache**]
 - > core [**turntable-core**]
 - > orm [**turntable-orm**]
 - > swagger2 [**turntable-swagger2**]

- > web [turntable-web]
- > weixin [turntable-weixin]
 - m pom.xml
- ▼ server [turntable-server]
 - > src
 - > target
 - m pom.xml
 - ⊗ .gitignore
 - m pom.xml
- > External Libraries
- > Scratches and Consoles

公众号 · 阿洋聊大数据

使用 maven-assembly-plugin 插件进行打包

在项目根目录下创建名为 `distribution` 子模块，在 `pom.xml` 添加服务入口 `jar` 依赖和 `maven-assembly-plugin` 插件。示例内容如下：

pom.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.
5 <parent>
6     <groupId>com.yzhou.wx</groupId>
7     <artifactId>turntable-springboot</artifactId>
8     <version>1.0-SNAPSHOT</version>
9 </parent>
10 <modelVersion>4.0.0</modelVersion>
11 <artifactId>turntable-distribution</artifactId>
12 <properties>
13     <maven.compiler.source>8</maven.compiler.source>
14     <maven.compiler.target>8</maven.compiler.target>
```

```
15     <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
16 </properties>
17 <dependencies>
18     <dependency>
19         <groupId>com.yzhou.wx</groupId>
20         <artifactId>turntable-server</artifactId>
21         <version>1.0-SNAPSHOT</version>
22     </dependency>
23 </dependencies>
24 <build>
25     <finalName>turntable-server</finalName>
26     <plugins>
27         <plugin>
28             <groupId>org.apache.maven.plugins</groupId>
29             <artifactId>maven-assembly-plugin</artifactId>
30             <configuration>
31                 <descriptors>
32                     <descriptor>assembly.xml</descriptor>
33                 </descriptors>
34                 <tarLongFileMode>posix</tarLongFileMode>
35             </configuration>
36             <executions>
37                 <execution>
38                     <id>make-assembly</id>
39                     <phase>install</phase>
40                     <goals>
41                         <goal>single</goal>
42                     </goals>
43                 </execution>
44             </executions>
45         </plugin>
46     </plugins>
47 </build>
48 </project>
49
```

在 pom.xml 同级目录下 创建 **assembly.xml** , 定义打包内容, 示例内容如下:

assembly.xml

```
1  <assembly>
2      <id>${project.version}</id>
3      <includeBaseDirectory>true</includeBaseDirectory>
4      <formats>
5          <format>dir</format>
6          <format>tar.gz</format>
7          <format>zip</format>
8      </formats>
9      <fileSets>
10         <fileSet>
11             <includes>
12                 <include>conf/**</include>
13             </includes>
14         </fileSet>
15         <fileSet>
16             <includes>
17                 <include>bin/*</include>
18             </includes>
19             <fileMode>0755</fileMode>
20         </fileSet>
21     </fileSets>
22     <files>
23         <file>
24             <!-- 打好的jar包名称和放置目录-->
25             <source>../server/target/turntable-server.jar</source>
26             <outputDirectory>target</outputDirectory>
27         </file>
28     </files>
29     <moduleSets>
30         <moduleSet>
31             <useAllReactorProjects>true</useAllReactorProjects>
32             <includes>
```

```
33         <include>com.yzhou.wx:turntable-server</include>
34     </includes>
35 </moduleSet>
36 </moduleSets>
37 </assembly>
```

创建 bin 目录 & 添加脚本

在 `distribution` 目录下创建 `bin` 目录，分别创建以下 4 个 file

```
1 shutdown.cmd
2 shutdown.sh
3 startup.cmd
4 startup.sh
```

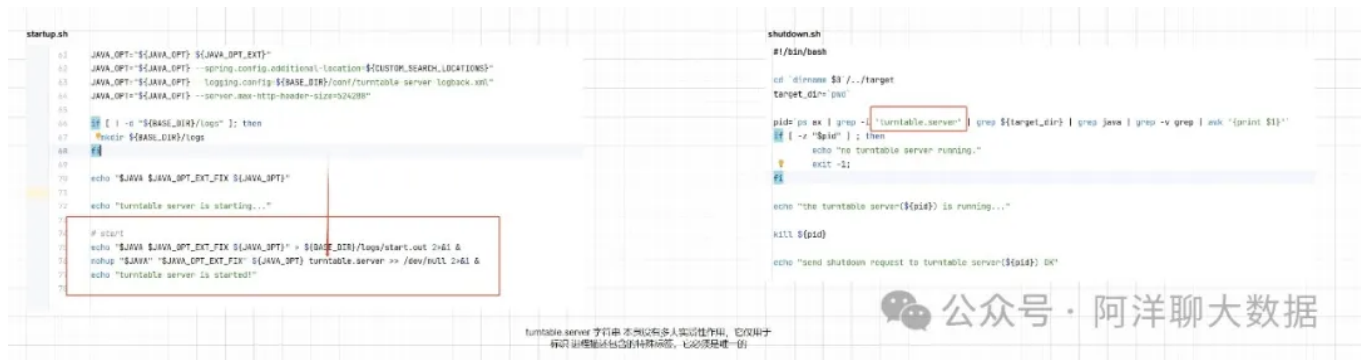
脚本中的一些文件名，变量名定义需要根据不同的服务名称做一些调整。这点非常重要 !!!

shutdown.cmd

```
1 @echo off
2 if not exist "%JAVA_HOME%\bin\jps.exe" echo Please set the JAVA_HOME variable
3 setlocal
4 set "PATH=%JAVA_HOME%\bin;%PATH%"
5 echo killing turntable server
6 for /f "tokens=1" %i in ('jps -m ^| find "turntable.server"') do ( taskkill /
7 echo Done!
```

shutdown.sh

shutdown 根据 `进程描述中的特殊符号` 来进行进程的筛选，再停止作业。我们必须保证它的 **唯一性**。



```
1 #!/bin/bash
2 cd `dirname $0`/../../target
3 target_dir=`pwd`
4 pid=`ps ax | grep -i 'turntable.server' | grep ${target_dir} | grep java | gr
5 if [ -z "$pid" ] ; then
6     echo "no turntable server running."
7     exit -1;
8 fi
9 echo "the turntable server(${pid}) is running..."
10 kill ${pid}
11 echo "send shutdown request to turntable server(${pid}) OK"
```

startup.cmd

```
1 @echo off
2 if not exist "%JAVA_HOME%\bin\java.exe" echo Please set the JAVA_HOME variabl
3 set "JAVA=%JAVA_HOME%\bin\java.exe"
4 setlocal enabledelayedexpansion
5 set BASE_DIR=%~dp0
6 rem added double quotation marks to avoid the issue caused by the folder name
7 rem removed the last 5 chars(which means \bin\) to get the base DIR.
8 set BASE_DIR="%BASE_DIR:~0,-5%"
9 set CUSTOM_SEARCH_LOCATIONS=file:%BASE_DIR%/conf/
10 set SERVER=turntable-server
11 set "APP_JVM_OPTS=-Xms512m -Xmx512m -Xmn256m"
12 rem set APP server options
13 set "APP_OPTS=%APP_OPTS% -jar %BASE_DIR%\target\%SERVER%.jar"
14 rem set APP server spring config location
```



```

15 set "APP_CONFIG_OPTS=--spring.config.additional-location=%CUSTOM_SEARCH_LOCA
16 rem set APP server log4j file location
17 set "APP_LOG4J_OPTS=--logging.config=%BASE_DIR%/conf/turntable-server-logback
18 set COMMAND="%JAVA%" %APP_JVM_OPTS% %APP_OPTS% %APP_CONFIG_OPTS% %APP_LOG4J_C
19 echo "turntable server is starting..."
20 rem start turntable server command
21 %COMMAND%
22 echo "turntable server is started!"

```

startup.sh

其脚本核心是通过 `--spring.config.additional-location` , `--logging.config` 设置启动参数。

```

39 export SERVER="turntable-server"
40
41
42 export JAVA_HOME
43 export JAVA="$JAVA_HOME/bin/java"
44 export BASE_DIR=`cd $(dirname $0)/../; pwd`
45 export CUSTOM_SEARCH_LOCATIONS=file:${BASE_DIR}/conf/
46
47 #=====
48 # JVM Configuration
49 #=====
50 JAVA_OPT="${JAVA_OPT} -Xms512m -Xmx512m -Xmn256m"
51
52 JAVA_MAJOR_VERSION=$(($JAVA -version 2>&1 | sed -E -n 's/.* version "([0-9]*).*$/\1/p')
53 if [[ "$JAVA_MAJOR_VERSION" -ge "9" ]] ; then
54     JAVA_OPT="${JAVA_OPT} -Xlog:gc*:file=${BASE_DIR}/logs/turntable_gc.log:time,tags:filecount=10,filesize=102400"
55 else
56     JAVA_OPT_EXT_FIX="-Djava.ext.dirs=${JAVA_HOME}/jre/lib/ext:${JAVA_HOME}/lib/ext"
57     JAVA_OPT="${JAVA_OPT} -Xloggc:${BASE_DIR}/logs/turntable_gc.log -verbose:gc -XX:+PrintGCDetails -XX:+PrintGCDateStamps -XX:+PrintGCTimeStamps -XX:+UseGCLogFile
58 fi
59
60 JAVA_OPT="${JAVA_OPT} -jar ${BASE_DIR}/target/${SERVER}.jar"
61 JAVA_OPT="${JAVA_OPT} ${JAVA_OPT_EXT}"
62 JAVA_OPT="${JAVA_OPT} --spring.config.additional-location=${CUSTOM_SEARCH_LOCATIONS}"
63 JAVA_OPT="${JAVA_OPT} --logging.config=${BASE_DIR}/conf/turntable-server-logback.xml"
64 JAVA_OPT="${JAVA_OPT} --server.max-http-header-size=524288"
65
66 if [ ! -d "${BASE_DIR}/logs" ]; then
67     mkdir ${BASE_DIR}/logs
68 fi
69
70 echo "$JAVA $JAVA_OPT_EXT_FIX $JAVA_OPT"
71
72 echo "turntable server is starting..."
73
74 # start
75 echo "$JAVA $JAVA_OPT_EXT_FIX $JAVA_OPT" > ${BASE_DIR}/logs/start.out 2>&1 &
76 nohup "$JAVA" "$JAVA_OPT_EXT_FIX" $JAVA_OPT turntable.server >> /dev/null 2>&1 &
77 echo "turntable server is started!"
78

```

- Line 40: `export SERVER="turntable-server"`
- Line 54: `JAVA_OPT="${JAVA_OPT} -Xlog:gc*:file=${BASE_DIR}/logs/turntable_gc.log:time,tags:filecount=10,filesize=102400"`
- Line 62: `JAVA_OPT="${JAVA_OPT} --spring.config.additional-location=${CUSTOM_SEARCH_LOCATIONS}"` (Annotated: `--spring.config.additional-location` 设置 conf 路径)
- Line 63: `JAVA_OPT="${JAVA_OPT} --logging.config=${BASE_DIR}/conf/turntable-server-logback.xml"` (Annotated: `--logging.config` 配置 logback 文件路径)

公众号 · 阿洋聊大数据

```

1 #!/bin/bash
2 cygwin=false
3 darwin=false
4 os400=false

```

```

5 case "`uname`" in
6 CYGWIN*) cygwin=true;;
7 Darwin*) darwin=true;;
8 OS400*) os400=true;;
9 esac
10 error_exit ()
11 {
12     echo "ERROR: $1 !!"
13     exit 1
14 }
15 [ ! -e "$JAVA_HOME/bin/java" ] && JAVA_HOME=$HOME/jdk/java
16 [ ! -e "$JAVA_HOME/bin/java" ] && JAVA_HOME=/usr/java
17 [ ! -e "$JAVA_HOME/bin/java" ] && unset JAVA_HOME
18 if [ -z "$JAVA_HOME" ]; then
19     if $darwin; then
20         if [ -x '/usr/libexec/java_home' ] ; then
21             export JAVA_HOME=`/usr/libexec/java_home`
22         elif [ -d "/System/Library/Frameworks/JavaVM.framework/Versions/CurrentJDK" ] ; then
23             export JAVA_HOME="/System/Library/Frameworks/JavaVM.framework/Versions/CurrentJDK"
24         fi
25     else
26         JAVA_PATH=`dirname $(readlink -f $(which javac))`
27         if [ "x$JAVA_PATH" != "x" ]; then
28             export JAVA_HOME=`dirname $JAVA_PATH 2>/dev/null`
29         fi
30     fi
31     if [ -z "$JAVA_HOME" ]; then
32         error_exit "Please set the JAVA_HOME variable in your environment, We
33     fi
34 fi
35 export SERVER="turntable-server"
36 export JAVA_HOME
37 export JAVA="$JAVA_HOME/bin/java"
38 export BASE_DIR=`cd $(dirname $0)/..; pwd`
39 export CUSTOM_SEARCH_LOCATIONS=file:${BASE_DIR}/conf/
40 #=====

```

```

41 # JVM Configuration
42 #=====
43 JAVA_OPT="${JAVA_OPT} -Xms512m -Xmx512m -Xmn256m"
44 JAVA_MAJOR_VERSION=$(($JAVA -version 2>&1 | sed -E -n 's/.* version "([0-9]*).
45 if [[ "$JAVA_MAJOR_VERSION" -ge "9" ]] ; then
46     JAVA_OPT="${JAVA_OPT} -Xlog:gc*:file=${BASE_DIR}/logs/turntable_gc.log:time
47 else
48     JAVA_OPT_EXT_FIX="-Djava.ext.dirs=${JAVA_HOME}/jre/lib/ext:${JAVA_HOME}/lib
49     JAVA_OPT="${JAVA_OPT} -Xloggc:${BASE_DIR}/logs/turntable_gc.log -verbose:gc
50 fi
51 JAVA_OPT="${JAVA_OPT} -jar ${BASE_DIR}/target/${SERVER}.jar"
52 JAVA_OPT="${JAVA_OPT} ${JAVA_OPT_EXT}"
53 JAVA_OPT="${JAVA_OPT} --spring.config.additional-location=${CUSTOM_SEARCH_LOC
54 JAVA_OPT="${JAVA_OPT} --logging.config=${BASE_DIR}/conf/turntable-server-logb
55 JAVA_OPT="${JAVA_OPT} --server.max-http-header-size=524288"
56 if [ ! -d "${BASE_DIR}/logs" ]; then
57     mkdir ${BASE_DIR}/logs
58 fi
59 echo "$JAVA $JAVA_OPT_EXT_FIX $JAVA_OPT"
60 echo "turntable server is starting..."
61 # start
62 echo "$JAVA $JAVA_OPT_EXT_FIX $JAVA_OPT" > ${BASE_DIR}/logs/start.out 2>&1
63 nohup "$JAVA" "$JAVA_OPT_EXT_FIX" $JAVA_OPT turntable.server >> /dev/null &
64 echo "turntable server is started!"

```

创建 conf 目录 & 添加配置文件

在 `distribution` 目录下创建 `conf` 目录，分别创建以下 2 个 file

```

1 application.yaml
2 turntable-server-logback.xml

```

turntable-server-logback.xml

```

1 <configuration>

```

```

2      <!-- 日志文件路径 -->
3      <springProperty scope="context" name="logFile" source="logging.file"/>
4      <property name="LOG_FILE" value="${logFile}"></property>
5      <!-- 日志打印级别 -->
6      <springProperty scope="context" name="logLevel" source="logging.level.root"/>
7      <property name="LOG_LEVEL" value="${logLevel}"></property>
8      <!-- 打印日志格式 -->
9      <property name="FILE_LOG_PATTERN"
10         value="${FILE_LOG_PATTERN:-%d{yyyy-MM-dd HH:mm:ss.SSS} ${LOG_LEVEL}%m%n"
11         <!-- %m输出的信息,%p日志级别,%t线程名,%d日期,%c类的全名,%i索引【从数字0开始递增】 -->
12         <!-- appender是configuration的子节点，是负责写日志的组件。 -->
13         <!-- ConsoleAppender：把日志输出到控制台 -->
14         <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
15             <encoder>
16                 <pattern>${FILE_LOG_PATTERN}</pattern>
17                 <!-- 控制台也要使用UTF-8，不要使用GBK，否则会中文乱码 -->
18                 <charset>UTF-8</charset>
19             </encoder>
20             <!-- 彩色日志 -->
21             <layout class="ch.qos.logback.classic.PatternLayout">
22                 <pattern>
23                     %d{yyyy-MM-dd HH:mm:ss} [%thread] %magenta(%-5level) %green(%-5msg) %n
24                 </pattern>
25             </layout>
26         </appender>
27         <!-- RollingFileAppender：滚动记录文件，先将日志记录到指定文件，当符合某个条件时
28         <!-- 以下的大概意思是：1.先按日期存日志，日期变了，将前一天的日志文件名重命名为XX
29         <!-- 2.如果日期没有发生变化，但是当前日志的文件大小超过1KB时，对当前日志进行分割
30         <appender name="SYSLOG" class="ch.qos.logback.core.rolling.RollingFileAppender">
31             <File>${LOG_FILE}</File>
32             <!-- rollingPolicy：当发生滚动时，决定 RollingFileAppender 的行为，涉及文件
33             <!-- TimeBasedRollingPolicy：最常用的滚动策略，它根据时间来制定滚动策略，
34             <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
35                 <!-- 活动文件的名字会根据fileNamePattern的值，每隔一段时间改变一次 -->
36                 <!-- 文件名：log/sys.2018-09-13.0.log -->
37                 <fileNamePattern>${LOG_FILE}-%d.%i.log</fileNamePattern>

```

```

38         <!-- 每产生一个日志文件，该日志文件的保存期限为30天 -->
39         <maxHistory>14</maxHistory>
40         <timeBasedFileNamingAndTriggeringPolicy class="ch.qos.logback.core.rolling.TimeBasedFileNamingAndTriggeringPolicy">
41             <!-- maxFileSize:这是活动文件的大小，默认值是10MB -->
42             <maxFileSize>64MB</maxFileSize>
43         </timeBasedFileNamingAndTriggeringPolicy>
44     </rollingPolicy>
45     <encoder>
46         <!-- pattern节点，用来设置日志的输入格式 -->
47         <pattern>${FILE_LOG_PATTERN}</pattern> <!-- 记录日志的编码 -->
48         <charset>UTF-8</charset> <!-- 此处设置字符集 -->
49     </encoder>
50 </appender>
51 <!-- 只用保存输出error级别的日志 -->
52 <appender name="ERRLOG" class="ch.qos.logback.core.rolling.RollingFileAppender">
53     <File>${LOG_FILE}_err.log</File>
54     <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
55         <FileNamePattern>${LOG_FILE}_err.%d.%i.log</FileNamePattern>
56         <maxHistory>14</maxHistory>
57         <timeBasedFileNamingAndTriggeringPolicy class="ch.qos.logback.core.rolling.TimeBasedFileNamingAndTriggeringPolicy">
58             <!-- maxFileSize:这是活动文件的大小，默认值是10MB -->
59             <maxFileSize>64MB</maxFileSize>
60         </timeBasedFileNamingAndTriggeringPolicy>
61     </rollingPolicy>
62     <encoder>
63         <!-- pattern节点，用来设置日志的输入格式 -->
64         <pattern>${FILE_LOG_PATTERN}</pattern> <!-- 记录日志的编码 -->
65         <charset>UTF-8</charset> <!-- 此处设置字符集 -->
66     </encoder>
67     <!-- 下面为配置只输出error级别的日志 -->
68     <filter class="ch.qos.logback.classic.filter.LevelFilter">
69         <level>ERROR</level>
70         <onMatch>ACCEPT</onMatch>
71         <onMismatch>DENY</onMismatch>
72     </filter>
73 </appender>

```

```

74     <!-- 控制台输出日志级别 -->
75     <root level="${LOG_LEVEL}">
76         <appender-ref ref="STDOUT"/>
77         <appender-ref ref="SYSLOG"/>
78         <appender-ref ref="ERRLOG"/>
79     </root>
80 </configuration>

```

application.yaml

`application.yaml` 是 spring boot 的配置文件，这里直接从 `resources` 目录下拷贝到 `conf` 目录即可。

这里需要注意，在 `turntable-server-logback.xml` 中定义了一些变量，例如 `logging.file` , `logging.level.root` 参数；

```

1     <springProperty scope="context" name="logFile" source="logging.file"/>
2     <property name="LOG_FILE" value="${logFile}"></property>
3     <!-- 日志打印级别 -->
4     <springProperty scope="context" name="logLevel" source="logging.level.root"/>
5     <property name="LOG_LEVEL" value="${logLevel}"></property>

```

那么在 `application.yaml` 中添加以下配置：

```

1 logging:
2   file: ${user.home}/logs/${spring.application.name}/${spring.application.name}.log
3   level:
4     root: INFO

```

到这里，我们需要做的配置。

总结

执行 `mvn clean install` 可对项目进行打包，在后面的服务启停，参数修改 效率都大大的提升。



YZhou

Kudos to the Author

[Read more](#)